from this set will be random.

The main takeaway here is that we can create a hash family that is 2-wise independent, but only requires a small amount of space to store and specify.

23.3 Static Dictionaries

In the static case, we can still use hashing with chaining, giving us an expected query time of $O(1)$. However, our goal is to have $O(1)$ query guaranteed.

To do this, we utilize *perfect hashing*: that is, an h such that $\forall k_1 \neq k_2$ in the database, then $h(k_1) \neq h(k_2)$.

The idea here is to use the "inverse" birthday paradox (an unofficial term); if there are very few people in a room, there probably are no two people that share the same birthday.

For our purposes, if the number of keys we have in our database is much smaller than the size of our database, then we're good.

Formally, let us look at the probability that there exists a collision for a random h from our hash family. We have

$$\mathbb{P}(\exists \text{collision}) = \mathbb{P}(\#\text{ collisions} \geq 1) \leq \frac{\mathbb{E}[\#\text{ collisions}]}{1}.$$

If we let Z_{ab} be the indicator for whether $h(j_a) = h(j_b)$, then the number of collisions is $\sum_{a<b} Z_{ab}$. As such, by linearity, we have

$$\mathbb{E}[\#\text{ collisions}] = \mathbb{E}\left[\sum_{a<b} Z_{ab}\right] = \sum_{a<b} \mathbb{E}[Z_{ab}] = \sum_{a<b} \mathbb{P}\big(h(j_a) = h(j_b)\big).$$

Since our hash family is 2-wise independent, this probability is just $\frac{1}{m}$. Further, there are $\binom{n}{2}$ elements in the set, so our final probability is $\frac{\binom{n}{2}}{m}$, and we know that this is $\leq \frac{1}{2}$ if we pick $m = n^2$.

This means that if we pick our hash table size to be at least n^2, then we'll probably not have any collisions. If we do have collisions, then we'll just keep trying other hash functions until no collisions happen (since the probability of no collision is at least $\frac{1}{2}$, this is just a Geom($\frac{1}{2}$) distribution, meaning we only need to try 2 times in expectation before we have no collisions—more importantly, this is a constant number of tries in expectation). After we find an h in pre-processing, we'd use this h to get our deterministic $O(1)$-time query.

12/1/2021

Lecture 24

Streaming Algorithms

There are two ways to view what streaming algorithms are.

- Algorithms that make one pass over data

  For example, with a search engine, we process the data as they come in.

- Dynamic data structures

  A data structural problem is one where we have some data and we want to efficiently layout the data to allow efficient queries. Dynamic data structures just mean that you can support insertion/deletion, etc.

In essence, when we refer to streaming algorithms, we care about minimizing memory usage. That is, we want $o(n)$ memory, or asymptotically less than linear memory.

As motivation, suppose we're looking at a network, and the packets sent over the network. TO analyze this data, we would like to not store every single packet that comes over the network; we'd like to have some algorithm that processes packets as they come through, and analyze and answer queries on the spot.

There are several types of streaming problems:

- Counting problems (which we'll cover today)
- Graph problems

  In a social network, the graph is continuously changing, and we'd like to query things like finding a path from one vertex to another, or finding a spanning forest, etc. It turns out that we can do things like create a spanning forest in $O(n\log^3 n)$ bits (which we won't be looking at today).
- Linear algebraic problems (ex. low-rank approximation, regression)
- Geometric problems
- Set maintenance

24.1 Counting

We want to maintain a counter N subject to three operations:

- `init()`: $N \leftarrow 0$
- `incr()`: $N \leftarrow N+1$
- `query()`: return N

We want to minimize memory usage.

The trivial algorithm is to maintain a single counter with N represented in binary (for example). This needs $\Theta(\log N)$ bits.

What if we know that our counter will never exceed t? Can we beat $\log t$ bits? No—it's impossible. Since we have t different numbers, we need at least t different counter values, and we need $\log t$ bits to store each of these counter values.

24.2 Approximate Counting

Suppose our counter value got to a really big number, i.e. 500 digits. Naively, this would require 500 digits of memory ($\approx 500\lceil\log_2 10\rceil$ bits).

As an approximation, suppose we just store $\lceil\log_{10} N\rceil$. If we want the answer to 1% error, we can store $\lceil\log_{1.01} N\rceil$.

Generalized, if we want to know the value up to $1+\varepsilon$ accuracy, the stored value will be $\Theta\left(\frac{\log N}{\varepsilon}\right)$, and we'd need $O(\log\log N - \log\varepsilon)$ bits, which is much better than the naive $\log N$ bits to store the exact number (which gets better for larger numbers).

However, this doesn't actually solve the problem; we don't know the number, so we can't increment anything. We don't know when we should go into the next number in the counter.

The idea here is to use randomness, and the algorithm we'll look at is Morris's algorithm.

(As an aside: why do we even care about these kinds of approximation counting algorithms, if we have lots of memory in modern-day computers? Morris, back in the 70's, wanted to keep track of occurrences of trigrams in a document (for spell-checking). Since there are approximately 26^3 different trigrams (more, including other characters), Morris found that he could only devote 8 bits per counter, but he needed to keep track of numbers more than 255; this is what led him to develop his algorithm, which we'll talk about today. However, a more modern application is with Redis, which is an in-memory database software. To implement caches (specifically, for the eviction policy), there needs to be a counter for how many times any given key was accessed, and this is proportional to the number of keys in the database, which could be incredibly large. Redis uses the Morris counter to keep track of those counters efficiently.)

A first idea is to not store N in memory, but instead store X. We'd have

- `init()`: $X \leftarrow 0$
- `incr()`: if `rand() % 2 == 0`, $X \leftarrow X+1$
- `query()`: return $2X$.

There are two problems here; we have a big error for small N (if we only increment once, then we either return 0 or 2, neither of which is within 1% of N), and we only save 1 bit of memory.

To save more memory, suppose we have

- `init()`: $X \leftarrow 0$
- `incr()`: with probability $\frac{1}{2^X}$, $X \leftarrow X+1$, otherwise do nothing
- `query()`: return $2^X - 1$

Here, it's very unlikely to increment X. To see why we return 2^X-1, we claim that $\mathbb{E}\left[2^{X_n}\right] = n+1$, where X_n is the value of X after the first n increments. If this is true, then we'd have $\mathbb{E}\left[2^X-1\right] = n+1-1 = n$, so this is an unbiased estimator for n.

To prove this claim, we proceed by induction on n. Our base case for $n=0$, we have $X_0 = 0$, and $2^{X_0} = 1 = n+1$.

In our induction step, we want to show the case for $n+1$. We have by total expectation

$$\begin{aligned}
\mathbb{E}\left[2^{X_{n+1}}\right] &= \sum_{j=0}^{\infty} \mathbb{E}\left[2^{X_{n+1}} \mid X_n = j\right] \mathbb{P}\left(X_n = j\right) \\
&= \sum_{j=0}^{\infty} \left(\frac{1}{2^j} \cdot 2^{j+1} + \left(1 - \frac{1}{2^j}\cdot 2^j\right)\right) \mathbb{P}\left(X_n = j\right) && \text{(total probability)} \\
&= \sum_{j=0}^{\infty} \left(2 + 2^j - 1\right) \mathbb{P}\left(X_n = j\right) \\
&= \sum_{j=0}^{\infty} \mathbb{P}\left(X_n = j\right) + \sum_{j=0}^{\infty} 2^j \, \mathbb{P}\left(X_n = j\right) \\
&= 1 + \mathbb{E}\left[2^{X_n}\right] && \text{(first term is summing over all possibilities of } X_n\text{)} \\
&= 1 + n + 1 = n+2
\end{aligned}$$

This then proves our claim for $n+1$.

To continue, we'd need to use Chebyshev's inequality, i.e. for all $\lambda > 0$,

$$\mathbb{P}(|X - \mathbb{E}[X]| > \lambda) < \frac{\text{Var}(X)}{\lambda}.$$

Another thing to keep in mind is that $\text{Var}(X) = \mathbb{E}\left[X^2\right] - \mathbb{E}[X]^2$.

Coming back to our problem, we have an estimator Z for N (for us, $Z = 2^X - 1$). We've already shown that $\mathbb{E}[Z] = N$.

As a next step, we'd like to determine

$$\mathbb{P}(|Z - N| > \varepsilon N) < \frac{\text{Var}(Z)}{\varepsilon^2 N^2}.$$

Our hope here is to have a small $\text{Var}(Z)$, i.e. we want $\frac{\text{Var}(Z)}{\varepsilon^2 N^2} < p$ for any p. If we did the calculation for the variance, we'd have that $\text{Var}(Z) = \frac{1}{2}N^2 - \frac{1}{2}N - 1$.

However, we have a problem here; our algorithm doesn't even take ε into account, so there's nothing to cancel out the ε^2. This means that we'd need to modify our algorithm to take ε into account, and get this variance down.

There are two ways to decrease the variance:

- We can run R independent copies of the algorithm in parallel, then use the average of all of the counters $Z = \frac{1}{R}\sum_{i=1}^{R} Z_i$ as our estimator.

  The expectation of this average is still N, but we have a lower variance, i.e. $\text{Var}\left(\frac{1}{R}\sum_{i=1}^{R} Z_i\right) = \frac{1}{R}\text{Var}(Z_i)$. If we choose $R = \frac{1}{\varepsilon^2 p}$, then we'd ensure that

  $$\mathbb{P}(|Z - N| > \varepsilon N) < \frac{\text{Var}(Z)}{\varepsilon^2 N^2} = \frac{\text{Var}(Z_i)}{R\varepsilon^2 N^2} \approx \frac{N^2}{\frac{1}{\varepsilon^2 p}\cdot \varepsilon^2 N^2} = p.$$

  Here, we used the fact that $\text{Var}(Z_i) \approx N^2$, as before.

- There are two observations we can make use of here. If we increment with probability 0.5^X, we have low memory, but high variance. On the other hand, if we increment with probability 1^X, then we have high memory, but low variance.

  Intuitively, there should be some continuous exchange between memory and variance as we change the probability. This means that if we increment with probability $\frac{1}{(1+a)^X}$, we can show that

  $$\mathbb{E}\left[\frac{(1+a)^X - 1}{a}\right] = N.$$

  This means that $Z = \frac{(1+a)^X - 1}{a}$ is our unbiased estimator. Further, we can also show that

  $$\text{Var}(Z) \le \frac{aN(N-1)}{2}.$$

  Letting $a \approx 2\varepsilon^2 p$, we'd get from Chebyshev's that $\mathbb{P}(|Z - N| > \varepsilon N) < p$.

  We can also show that the memory consumption is close to $O(\log\log N + \log\frac{1}{a})$, and with our choice of a, we need $O(\log\log N + \log\frac{1}{\varepsilon} + \log\frac{1}{p})$ bits.

24.3 Unique Counting

The distinct/unique counting problem is similar to the generic count problem, except we want to count the number of distinct items we see, in a stream of m items (possibly with repeats) coming from a universe of size $\{1, \ldots, n\}$.

There exists an algorithm using $O(\frac{1}{\varepsilon^2} + \log n)$ bits of memory with success probability of 99%. We can also prove that there is no better algorithm; this is optimal.

We won't be looking at this specific algorithm today, but we'll briefly go over an idealized version of the algorithm, and explain its connection to hashing.

We have `init()` that sets $X \leftarrow 1$, and we pick $h : [n] \to [0, 1]$, i.e. h maps $[n]$ to a real number between 0 and 1.

We have `update(i)` as $X \leftarrow \min(X, h(i))$. That is, we'll always be storing the smallest hash value seen so far. It can be shown that the expected value of X is equal to $\frac{1}{N+1}$ (since in expectation the hashes should be equally spaced in $[0, 1]$).

This means that our `query()` would need to be $\frac{1}{X} - 1$ in order for $\mathbb{E}[Z] = \mathbb{E}\left[\frac{1}{X} - 1\right] = N$, our true unique count.

There are a few problems with this. Firstly, we can't store exact real numbers, so we'd need to store them up to some finite precision. This can be done in a straightforward manner, but the more pressing problem is our hash function itself.

We'd need to store this hash function in order to maintain consistency, but this hash function is just a mapping for all n possible inputs, i.e. we need $O(n)$ memory. This is really bad, since the naive solution already takes $O(n)$ memory. As such, we can replace h with a hash function from a 2-wise independent family, and reduce the memory needed to store the hash function.

This is another example of how we can use 2-wise independent families to reduce the memory consumption of storing the hash function in memory.